Claude Code client guide — install, sign-in & configuration

How Claude Code is installed, signed in, and configured on developer laptops.

- **Part I (§1–§5) is the developer user manual**: prerequisites, install, the sign-in walkthrough as it actually behaves, and troubleshooting. It is written to be handed to end users as-is.
- **Part II (§6–§9) is for administrators**: what the gateway pushes centrally, which settings must come from a managed source, and the GPO/MDM delivery of the login policy.

The rollout model in one line: the download portal's installer ZIP (`client/Install-ClaudeCode.ps1` under the hood) installs the binary and writes workstation config **entirely in user scope** — no admin rights — but **gateway login requires one admin-delivered managed setting**: Claude Code only offers the "Cloud gateway" login when it is present, by Anthropic's design (§8). That setting is delivered by GPO/MDM (the AD request is `ad-request-email.md`) or self-served by a developer with local admin. So the model is *no-admin install + one required managed policy for login*.

This is an operations how-to and is deliberately **not** part of the PDF review package — but Part I is rendered on its own to `../generated/user-manual.pdf` as the end-user handout (regenerated by `make docs-pdf`; edit this file, never the PDF). The ConOps (`conops.md`) references this model; `../ato/security-assessment-2026-07.md` records the control rationale.

Part I — Developer user manual

1. Prerequisites

Before installing and running Claude Code you need:

- **Git for Windows — required** (Windows laptops). Claude Code's shell tooling runs through Git's `bash.exe`, and its version-control features (diffs, commits, branches) use `git` itself. A standard install is found automatically (Claude Code looks in `C:\Program Files\Git\bin\bash.exe` and `C:\Program Files (x86)\Git\bin\bash.exe`). If your Git lives anywhere else — a per-user or portable install — set the `CLAUDE_CODE_GIT_BASH_PATH` environment variable to your `bash.exe` location (see §5.1).
- **On Linux:** a glibc x86-64 distribution (Ubuntu/Debian/RHEL and derivatives — the portal serves the `linux-x64` glibc build, not the Alpine/musl one), with `git`, `bash`, and the `sha256sum` coreutils (present on any standard developer distribution); `python3` is used by the installer to write your settings file — without it the binary still installs and the installer prints the settings to add by hand.
- **Node.js / npm — NOT required.** This deployment distributes the precompiled native binary (`claude.exe` on Windows, `claude` on Linux); nothing is installed from npm, and `npm install`-based instructions you may find online do not apply here (updates are locked down and this network does not reach npm anyway).

- **An Okta account** in the authorized group, with MFA enrolled — the same corporate SSO sign-in gates both the download portal and Claude Code itself.
- **The managed login policy on your machine.** IT delivers it automatically by GPO/MDM; there is nothing for you to do. Without it Claude Code has no gateway login option at all — if `/login` shows no "Cloud gateway" screen, contact IT rather than trying to configure it yourself (a developer with local admin can self-serve it — §8).
- **The gateway certificate fingerprint published by IT** — you confirm it once, at first connect. The download portal also shows the current value live on its **Gateway fingerprint** page (§4.4).
- **No admin rights** — the install itself is entirely user-scope.

2. Installing

2.1 Downloading from the portal

The installer comes from the **download portal**: open the portal link IT published (`https://<gateway-host>/portal`) in your browser and sign in with your normal Okta SSO. The portal opens on a home page of cards — the portal is also where you can check your usage and quotas, read this guide, and see the gateway fingerprint (§4) — pick **Download installer**. Before the download, the page asks for three picks:

1. **Cost center** — choose yours first.
2. **Team** — the team list fills in with only the teams belonging to the cost center you picked. Choose the team you actually work in.
3. **Platform** — **Windows (64-bit)** or **Linux (x64)**, then start the download.

(If your browser has scripts disabled, the same page works as two quick page loads instead: pick the cost center, continue, then pick the team and platform.)

What the picks are for, and how to choose. Your selections are baked into the installer and become labels on your usage telemetry (`cost_center=…`, `team=…`), so your Claude Code usage and spend are attributed to the right team and cost center on IT's dashboards. They are **attribution only** — they grant nothing and restrict nothing: no permissions, models, or limits depend on them. Choose the team you actually work in; if you don't know which cost center funds your work, ask your manager or IT rather than picking the closest match — a wrong pick means your usage shows up under someone else's budget. Both lists (and which teams belong to which cost center) are maintained by IT, so if yours is missing, tell IT — don't substitute another. Picked wrong? Download again with the right pair and re-run the installer: re-installing is harmless and simply overwrites the labels. Each download is recorded (who, which team/cost center, which version) in an audit log.

2.2 Running the installer (Windows)

The ZIP contains `claude.exe`, the installer script (`Install-ClaudeCode.ps1`), an `install.cmd` with your portal picks and the deployment's standard options baked in, a `README.txt`, and — when the deployment bundles one — an `extra-ca.pem` trust file. Extract it and double-click `install.cmd`. No elevation prompt appears; the installer does exactly four things, all in your own profile:

- **Binary** → `%USERPROFILE%\.local\bin\claude.exe`, verified (SHA-256 against the release manifest + Anthropic Authenticode) on a local staging copy before it is moved into place.
- **User PATH** → `%USERPROFILE%\.local\bin` is appended to the *user* `Path` environment variable (registry-backed, persists; no machine PATH edit).
- **User configuration** → an `env` block merged into `%USERPROFILE%\.claude\settings.json` (your own settings file). The merge preserves every existing top-level key and every unrelated `env` key, and refuses to overwrite a file it cannot parse. The keys written:

Key (under `env`)	Set by	Purpose
`DISABLE_UPDATES = 1`	`-DisableUpdates`	Blocks all update paths (background checks **and** manual `claude update` / `claude install`) — keeps users on the distributed build
`DISABLE_AUTOUPDATER = 1`	`-DisableUpdates`	Background-check lockdown, defense in depth
`OTEL_RESOURCE_ATTRIBUTES`	`-Team` / `-CostCenter` (your portal picks, passed via `install.cmd`)	Telemetry grouping labels (`team=…`, `cost_center=…`); telemetry itself is enabled centrally by the gateway
`NODE_EXTRA_CA_CERTS`	`-ExtraCaCertPath`	Enterprise CA trust for the gateway TLS chain (the precompiled binary honors it)

- **Onboarding flag** → `hasCompletedOnboarding: true` is set in `%USERPROFILE%\.claude.json` (created if the machine has no Claude config at all; existing keys are preserved, an unparseable file is never overwritten). Without it, a first-ever run tries to reach Anthropic's public endpoints — blocked on this network — and exits with "Unable to connect to Anthropic services" before the gateway login appears (§5.6).

These are ordinary environment variables, honored from the user settings file — **not** policy keys. The installer never writes `%ProgramFiles%\ClaudeCode\managed-settings.json` and never touches `HKx\SOFTWARE\Policies\ClaudeCode`. A SYSTEM-context run is refused outright.

After installing, **open a new terminal** so the updated PATH is picked up.

2.3 Running the installer (Linux)

The Linux ZIP contains the `claude` binary (linux-x64), the installer script (`install-claude-code.sh`), an `install.sh` with your portal picks and the deployment's standard options baked in, a `README.txt`, and — when the deployment bundles one — an `extra-ca.pem` trust file. Unzip it and run:

```
unzip claude-code-<version>-linux.zip -d claude-code && cd claude-code
bash install.sh
```

No root or sudo is involved; the installer is the Linux twin of the Windows one and does the same four user-scope things:

- **Binary** → `~/.local/bin/claude` (mode 0755), verified by SHA-256 against the release manifest on a local staging copy before it is moved into place. (There is no Authenticode on Linux; the checksum chain back to the GPG-verified release manifest is the integrity check.)
- **PATH** → if `~/.local/bin` is not already on your `PATH` (most distributions put it there), a guarded `export PATH=…` line is appended to `~/.bashrc`. If your shell is not bash, add the equivalent line to your shell's profile yourself.
- **User configuration** → the same `env` block as §2.2's table, merged into `~/.claude/settings.json` with the same rules (preserves unrelated keys, refuses to overwrite a file it cannot parse). The one path difference: a bundled enterprise CA is copied to `~/.local/bin/claude-extra-ca.pem` and referenced from `NODE_EXTRA_CA_CERTS`.
- **Onboarding flag** → `hasCompletedOnboarding: true` is set in `~/.claude.json`, same rules and rationale as §2.2's onboarding bullet (§5.6 has the failure this prevents).

The installer refuses to run as root (it would install into root's home, not yours). After installing, **open a new terminal** and continue with §3 — the sign-in flow is the same on Linux, and the login policy your machine needs is the Linux managed-settings file (§8.5), delivered by IT.

3. First launch & signing in

Signing in is effectively a one-time step: the session persists and refreshes itself. The login method is locked to the corporate gateway and the URL is pre-filled — you never choose an account type or type a URL.

1. Open a **new** terminal and run `claude`.
2. Two things can happen here, **both normal**:
3. Claude Code opens directly on the **gateway login screen**, or
4. it drops you into the **chat window** with a notice telling you to run `/login`. Type `/login` and the same gateway screen appears.
5. The gateway screen shows the pre-filled gateway URL. Press **Enter** to accept and connect.
6. Claude Code usually does **not** launch your browser by itself. Copy the authentication URL it prints and paste it into your browser, then complete the Okta sign-in (+ MFA).
7. At first connect Claude Code also shows the gateway certificate fingerprint and asks you to confirm it (trust-on-first-use). Compare it against the fingerprint IT published — the portal's **Gateway fingerprint** page (§4.4) shows the current value. Do not blind-accept it; that prompt is what detects TLS interception in front of the gateway.
8. Back in the terminal, Claude Code tells you to press Enter to continue. About half the time the Enter key does nothing at this point — just **close the command window, open a new one, and run** `claude`: you will be fully signed in.
9. Occasionally Claude Code comes back from a successful sign-in not responding to the keyboard at all. Close it and reopen it — the session is already saved.
10. If you have used Claude in some other form before (claude.ai, a personal API key, an earlier install), you may see a **yellow warning** that a previous model was not found and that you are now using **Opus 4.8**. This is normal — see §5.7.

Signing out (if you ever need to): use `claude auth logout` — never the `/logout` slash command. On this network `/logout` leaves Claude Code unable to start (§5.6 has the recovery).

4. Day-to-day notes

4.1 Everyday behavior

- `/model` lists exactly three models — Opus 4.8 (the default), Sonnet 5, and Sonnet 4.5. That is by design, not an error; nothing else is served here.
- WebSearch and MCP tools are disabled centrally and cannot be re-enabled in your local settings. WebFetch works: it fetches from your laptop, subject to the same web access policy as your browser.
- Updates are disabled; you always run the IT-distributed build, so `claude update` doing nothing is intentional. New versions are published to the download portal (the version is shown on its page) — when IT announces one, download a fresh installer ZIP and run `install.cmd` again.
- `/status` shows which configuration sources are active — useful to see what is centrally managed vs. your own `settings.json`.

The download portal (`https://<gateway-host>/portal`, same Okta sign-in as the installer download) is also your self-service window into the deployment — the three pages below.

4.2 My usage & quotas

The portal's **My usage & quotas** page shows your own Claude Code spend: for each budget period that applies to you (daily, weekly, or monthly) it lists your spend cap, your spend so far this period, and the percentage used, with a colored progress bar. It also tells you where each cap comes from — a cap set for you personally, one inherited from an Okta group you are in, or the organization-wide cap. If no cap applies to you, the page still shows your spend, marked "no cap".

Notes:

- Hitting a cap does not break anything permanently — Claude Code starts refusing requests with a "spend limit reached" message until the period rolls over or an admin raises the cap. This page is how you see the cap coming before you hit it.
- Spend figures come from the gateway's own metering, the same numbers cap enforcement uses.
- If the page says usage display is not enabled on this deployment, that is a deployment-side setting — mention it to IT if you expected to see your numbers.

4.3 The user guide

The portal's **User guide (PDF)** page shows this manual: read it in the browser or use its download link to save a copy. The portal always serves the version IT most recently published, so prefer it over stale local copies.

4.4 Gateway fingerprint

The portal's **Gateway fingerprint** page shows the gateway's current TLS certificate fingerprint, fetched live from the gateway itself. It is printed in the two common formats — uppercase pairs separated by colons (`AA:BB:…`) and the same value as one lowercase hex string — so whichever style Claude Code's first-connect prompt uses, you can compare directly (§3 step 5).

If the fingerprint Claude Code shows you matches neither form, **stop — do not accept it** — and contact IT: a mismatch can mean something is intercepting TLS between your machine and the gateway. Also expect the value to change when IT rotates the gateway certificate; the portal page always shows the current one.

5. Troubleshooting

5.1 Claude Code can't find Git / bash

Symptoms: a startup error saying Claude Code requires Git for Windows, shell commands failing to run, or *"Claude Code was unable to find CLAUDE_CODE_GIT_BASH_PATH path …"*.

- If Git for Windows is not installed, install it (no admin needed for the per-user installer).
- If Git is installed somewhere other than the standard `C:\Program Files\Git` location (per-user install, portable Git), point Claude Code at your `bash.exe` with the `CLAUDE_CODE_GIT_BASH_PATH` environment variable:

```
powershell setx CLAUDE_CODE_GIT_BASH_PATH "D:\Tools\Git\bin\bash.exe"
```

then open a **new** terminal. Alternatively, put it in the `env` block of `%USERPROFILE%\.claude\settings.json` alongside the installer's keys:

```
json { "env": { "CLAUDE_CODE_GIT_BASH_PATH": "D:\\Tools\\Git\\bin\\bash.exe" } }
```

The value must be the full path to `bash.exe` **itself** (normally `…\Git\bin\bash.exe`), not the Git folder and not `git.exe`.

5.2 No login screen at launch — dropped into the chat window

Sometimes the first launch does not take you to the login screen and instead opens the chat window with a notice to log in. This is normal: run `/login` and continue from §3 step 3.

If `/login` shows **no gateway screen at all** (an account picker with no "Cloud gateway" option), the managed login policy is missing from the machine — contact IT (§8; there is nothing you can set in user scope to fix this).

5.3 The browser never opens during sign-in

Expected most of the time: after you accept the gateway screen, Claude Code typically does not launch the browser itself. Copy the authentication URL from the terminal and paste it into your browser manually.

5.4 "Press Enter to continue" does nothing

After the browser sign-in completes, roughly half the time the terminal's Enter key has no effect on the "press Enter to continue" prompt. Close the command window, open a new one, and run `claude` — you will be fully signed in (the session was already saved).

5.5 Keyboard input not recognized after signing in

Occasionally, after a successful sign-in, Claude Code stops responding to keyboard input entirely. Close Claude Code and reopen it; the session persists, so you will not have to sign in again.

5.6 Startup fails: "Unable to connect to Anthropic services"

Claude Code exits at launch before any login screen appears. This happens when the onboarding flag is unset — after running the `/logout` slash command, or on a profile that was never touched by the installer — because a not-yet-onboarded client tries to reach Anthropic's public endpoints, which this network blocks by design. The installer sets this flag for you at install time, so on a fresh install you should not see this error; if you do, or you ran `/logout`, fix it as follows.

Fix (no admin needed): mark onboarding as completed in `%USERPROFILE%\.claude.json`:

```
$p = "$env:USERPROFILE\.claude.json"
Copy-Item $p "$p.bak"          # this file also holds project history — back it up
$j = Get-Content $p -Raw | ConvertFrom-Json
if ($j.PSObject.Properties.Name -contains 'hasCompletedOnboarding') {
  $j.hasCompletedOnboarding = $true
} else {
  $j | Add-Member -NotePropertyName hasCompletedOnboarding -NotePropertyValue $true
}
$j | ConvertTo-Json -Depth 100 | Set-Content $p -Encoding utf8
```

(If `%USERPROFILE%\.claude.json` does not exist yet, create it containing just `{"hasCompletedOnboarding": true}`.)

On Linux the same flag lives in `~/.claude.json`; back the file up (`cp ~/.claude.json ~/.claude.json.bak`) and set it with:

```
python3 - <<'EOF'
import json, os
```

```
p = os.path.expanduser("~/.claude.json")
d = json.load(open(p)) if os.path.exists(p) else {}
d["hasCompletedOnboarding"] = True
json.dump(d, open(p, "w"), indent=2)
EOF
```

Then open a **new** terminal, run `claude`, and run `/login`. Because `/logout` also deletes the pinned certificate fingerprint, the fingerprint confirmation from §3 step 5 will come back — re-check it against the IT-published value. To avoid the whole trap, sign out with `claude auth logout`, never `/logout`. The service-desk version of this procedure is `om-runbooks.md` runbook 12.

5.7 Yellow warning: a model was not found — now using Opus 4.8

If you previously used Claude in some other form (claude.ai, a personal API key, another workplace's install), your old settings may name a model this gateway does not serve. On login Claude Code warns in yellow that the model was not found and switches you to **Opus 4.8**. This is normal behavior, not an error — Opus 4.8 is the default here, and `/model` shows what you can switch to.

5.8 Startup fails: your Claude Code version is below the minimum

The gateway enforces a minimum Claude Code version (normally the gateway's own version). If your installed client is older, Claude Code exits at startup with a message saying the version is below the required minimum. The built-in updater is disabled on this deployment, so the fix is to download and run the latest installer from the portal (§2.1) — no admin rights needed, same as the first install. Your sign-in, settings, and history are kept. You'll hit this after the platform team upgrades the gateway; the download portal always has the matching installer.

5.9 Bounced to the browser sign-in every hour

If you are pushed through the browser SSO at every session expiry (about hourly) and `/login` then shows the default picker until you restart Claude Code, report it to your admins — it means the Okta app is missing the **Refresh Token** grant type (so the session cannot refresh silently). Admin fix: `okta-request-email.md` and `om-runbooks.md`. It is not a problem with your machine or settings.

Part II — Administrators

6. What the gateway pushes centrally

After a client authenticates, the **gateway pushes settings to it** via its `/managed/settings` endpoint — the same mechanism it already uses to hand clients their telemetry (OTLP) configuration.

Nine things are pushed:

a) The model allowlist — always, to every user. The gateway pushes `availableModels: [<OPUS_MODEL_ID>, <SONNET_MODEL_ID>, <HAIKU_MODEL_ID>]` plus `enforceAvailableModels: true`, which is what constrains the `/model` picker to the three configured models. This policy carries **no** `match:`, so it applies to every authenticated user and needs no Okta groups claim.

> **This is load-bearing, not cosmetic.** `models:` in the gateway config only controls what the *gateway serves* — it does not reach into the client's picker. Without the allowlist push, `/model` shows Claude Code's own built-in menu, none of whose entries this gateway serves, so every selection fails upstream as unauthorized.

Both keys are ordinary Claude Code `settings.json` keys and therefore belong **inside the policy's `cli:` object** — the blob the gateway forwards as the client's managed settings. `availableModels` at the *policy* level is an unrecognized key that fails gateway config validation and prevents boot.

`cli` contents are **not** checked when the config loads, but they are strictly validated against the CLI settings schema before being served — an unknown key there is rejected with *"unknown settings key — fix the typo, upgrade the gateway if this key was added in a newer CLI…"*. So a typo inside `cli` survives startup and surfaces later, on the `/managed/settings` path. Do not treat `cli` as a free-form passthrough.

Policy order is load-bearing. Selection is first-match-wins over a *single* policy, and a policy with no `match:` is normalized to `match: {}`, which matches everyone — so the allowlist policy **must be last**, or every policy after it is dead config. With it last, the gateway merges its `cli` as a *base* into each earlier policy, so group members receive the allowlist *and* the lockdown. `availableModels` accepts family aliases (`opus`), version prefixes (`opus-4-5`), and full model IDs; an empty array means "default model only".

b) Update lockdown — also to everyone. The same catch-all policy carries `DISABLE_UPDATES` / `DISABLE_AUTOUPDATER` as `cli.env`, the server-side twin of the installer's `-DisableUpdates`. It is deliberately unscoped: pushing the lockdown to every authenticated user is broader coverage than a group-scoped push and drops a dependency on the Okta groups claim.

c) Web/MCP tool denies — also to everyone. The policy carries `permissions: { deny: ["WebSearch", "mcp__*"] }` in `cli`. A bare tool name in `deny` removes the tool from the model's context entirely — stronger than a scoped deny like `WebFetch(domain:*)`, which only blocks matching calls — and `mcp__*` covers every tool of every MCP server. Deny rules union across scopes and a deny at any scope cannot be re-allowed at another, so users cannot re-enable any of them locally. Rationale per tool:

- **WebSearch** executes *server-side* and **Amazon Bedrock does not expose the server-side web search tool at all**, so it is non-functional on this gateway regardless — the deny is defense-in-depth tidiness.
- `mcp__*` closes the remaining tool-borne web path (an MCP server can expose fetch-like tools). The offline installer also controls what MCP config users receive, so this is belt-and-braces.

Two tools are deliberately **not** denied. **WebFetch** (allowed since 2026-08-07; it was denied in the original rollout) fetches *locally on the client* — the CLI does the HTTP request, after a hostname-only preflight to `api.anthropic.com` — so it is bounded by the same Zscaler client-side web policy as the developer's browser.

Subagents (`Agent`) stay allowed because a bare `Agent` deny would block all subagents, built-in and custom alike. The model can likewise reach the web through `Bash` (`curl`/`wget`), bounded by the same Zscaler policy; a `Bash(curl *)`-style deny was considered and not applied.

d) The small/fast model override — also to everyone. The policy sets `env.ANTHROPIC_DEFAULT_HAIKU_MODEL` to the configured haiku-role model ID, `<HAIKU_MODEL_ID>` (Sonnet 4.5 by default — it moved into this slot when Sonnet 5 became the Sonnet tier). Claude Code uses a Haiku-family model for lightweight background tasks by default — but no Haiku model exists in GovCloud and this gateway does not serve one, so without the override those background calls request an unserved model and fail. The value is the same `<HAIKU_MODEL_ID>` as the allowlist — the **gateway-facing** model ID, *not* the Bedrock inference-profile ID (`HaikuBedrockModelId`): the client asks the gateway, and the gateway's `models:` block does the Bedrock mapping. The client uses the env var's value verbatim — no rewriting or region logic — and the env-var path performs no allowlist check, so `enforceAvailableModels` cannot block it; the value is allowlisted anyway. Two caveats: `ANTHROPIC_SMALL_FAST_MODEL` is the deprecated name for the same knob and **takes precedence if set locally** on a user's machine (its `_AWS_REGION` companion is Bedrock-direct-only and irrelevant behind a gateway); and the picker may show a cosmetic "Custom Haiku model" entry for the override — it resolves to an allowlisted model either way.

e) The minimum client version — also to everyone. The policy carries `requiredMinimumVersion` in `cli` (rendered when the `MinClientVersion` stack parameter is set; `deploy-gateway.sh` defaults it to `CLAUDE_VERSION`, the version the gateway image itself runs, so the floor follows every gateway upgrade — override with `MIN_CLIENT_VERSION` in `deploy.env`, or set it to `none` to disable). A client running an older version **exits at startup** with instructions to update. Three properties to know:

- Enforcement is a **client-side startup check** honored only from managed (policy) settings — which is what the gateway push becomes on the client. A client learns the floor when it fetches `/managed/settings`, so an out-of-date client keeps its *current* session and is blocked at its **next start** after a fetch: a ratchet for the installed base, not an instant gate.
- It **fails open by design**: an invalid value is stripped rather than enforced, so a bad push cannot brick startup fleet-wide (per the settings documentation; `deploy-gateway.sh` and the template's `AllowedPattern` reject non-`X.Y.Z` values before they ship anyway).
- Because auto-updates are locked down (b) and updates flow through the portal, raising the floor **must trail publishing the matching installer** (`publish-portal-release.sh`) — the startup error tells users to update, and the portal is where they can actually do it.

f) Prompt / response content capture — only when the organization enables it. Two independent `deploy.env` opt-ins (both off by default): `LOG_USER_PROMPTS=true` adds `env.OTEL_LOG_USER_PROMPTS: "1"`, and each client's `claude_code.user_prompt` telemetry event then carries the **full text of every prompt the user types** (by default only the prompt's length is reported); `LOG_ASSISTANT_RESPONSES=true` adds `env.OTEL_LOG_ASSISTANT_RESPONSES: "1"`, and the `claude_code.assistant_response` event carries the **model's response text** (honored by clients ≥ 2.1.193). The content flows into the organization's activity audit stream — per-user attributed, CMK-encrypted, IAM-only — alongside the bash-command/tool-input records; `deploy-gateway.sh` refuses either flag unless that stream (`FORWARD_ACTIVITY_LOGS=true`) is on. When prompts are captured but responses are not, the policy pins `OTEL_LOG_ASSISTANT_RESPONSES: "0"` explicitly, because that setting otherwise falls back to the prompts flag. Like every managed push, both take effect at the client's next settings fetch. If your organization enables either, say so in your acceptable-use / rules-of-behavior material — users should not have to discover from a diagram that their prompts or the model's answers are retained.

g) Organization-wide Claude rules — only when the organization provides them. Point `CLAUDE_MD_FILE` in `deploy.env` at a markdown rules file (start from `scripts/claude-rules.example.md`) and re-run `deploy-gateway.sh`: the content is pushed to every client as the `claudeMd` managed-settings key. The client loads it into **every session's context**, ahead of the user's own `~/.claude/CLAUDE.md` and any project `CLAUDE.md` (which add to it), and — like all managed memory — users **cannot exclude or override it**. Use it for short, cross-team engineering rules (secrets handling, verification discipline, test-before-commit); it is a prompt-level control, not a technical enforcement mechanism like the tool denies in (c). Four properties to know:

- **Keep it short.** The text lands in every session for every user, so its length is a per-request context cost across the whole fleet. `deploy-gateway.sh` enforces the hard bound (4,096 characters JSON-encoded, a CloudFormation parameter limit — just under 4 KB of typical markdown, since newlines, quotes, and backslashes each encode to two characters); well under that is better.
- The deploy script JSON-string-encodes the file so arbitrary markdown survives the template rendering; never paste raw markdown into the `ManagedClaudeMd` stack parameter directly (the parameter's `AllowedPattern` rejects it).
- **The rules text must not contain a `${` sequence.** The gateway expands `${NAME}` in its config values as *environment variables* after YAML parsing — an undefined name fails the gateway boot, a defined one is silently substituted into the rules text, and no escape syntax exists (all verified against the 2.1.211 gateway binary). `deploy-gateway.sh` and the parameter pattern both refuse the sequence; write `$NAME`, `$ {`, or prose instead. Everything else — quotes, backslashes, bare `$`, em-dashes, emoji — round-trips verbatim.
- Like every managed push, changes reach clients at their **next settings fetch** — no reinstall, no portal release.
- Gateway-side behavior is binary-verified (the mirrored 2.1.211 gateway boots with the key and serves the content verbatim at `/managed/settings`); the client rendering the rules into context is verified against the client memory documentation but **needs live confirmation** — after first enabling it, run `/memory` on one client and confirm the managed rules appear.

h) Session-recap disable — only when the organization opts in. Set `DISABLE_SESSION_RECAPS=true` in `deploy.env` and re-run `deploy-gateway.sh`: the policy carries `awaySummaryEnabled: false` in `cli`, and clients stop showing session recaps — both the recap shown when a user returns to a session after being away and the remote-recap variant, whose client-side gate checks the same key. The default (`false`) pushes nothing, leaving the client default (recaps on). Gateway-side behavior is binary-verified (the mirrored 2.1.211 and 2.1.220 gateways boot with the key); the client honoring it is verified against the client settings schema but **needs live confirmation** — after first enabling it, leave a pilot session idle past five minutes (the threshold the client settings schema states for the recap) and confirm no recap appears.

i) Enterprise skills — only when the organization hosts them. There is **no direct skill-push key** in the managed settings; skills ship **inside a plugin** from a marketplace the organization hosts. The push has two keys in `cli`, each with its own `deploy.env` switch:

- `extraKnownMarketplaces` (from the `PLUGIN_MARKETPLACE_*` variables) registers the marketplace on every client: a github `owner/repo` or a full git URL (internal GitLab/Gitea over https/ssh both work), optionally pinned to a branch/tag. Start the repository from `scripts/enterprise-marketplace.example/`.
- `enabledPlugins` (from `MANAGED_PLUGINS`, comma-separated plugin names) force-installs plugins from that marketplace at **managed scope**: users cannot disable them, and every skill inside becomes available in each session (invoked as `/<plugin>:<skill>`, or triggered by the model from the skill's description; a skill's body loads into context when it triggers, not up front). `PLUGIN_MARKETPLACE_NAME` must equal the `name` field in the marketplace repo's `marketplace.json` — a mismatch is untested client behavior; keep them identical.

Three properties to know. First, **clients fetch the marketplace directly** — the gateway only pushes the pointer, and the offline build host is never involved — so the marketplace host must be reachable from developer laptops (Zscaler policy, internal DNS), **and readable without interactive git authentication**: the client fetches with an internal git implementation that does not use git credential helpers, `gh` auth, or OS keychains, so a private github.com repo fails outright (anthropics/claude-code#17201). Use a repo that is anonymously readable from the corporate network (an internal git server is usually the right answer). Second, with `PLUGIN_MARKETPLACE_AUTO_UPDATE=true` (the default) clients refresh the marketplace and its plugins at startup, so **pushing to the marketplace repo is the whole release process** for a skill update (bump `version` in the plugin's `plugin.json`). Third, the verification status: gateway-side behavior is binary-verified (the mirrored 2.1.211 and 2.1.220 gateways boot with both keys and the exact JSON value shapes the deploy renders), but **client-side auto-install is contested and needs live confirmation** — a public tracker report (anthropics/claude-code#45323, closed unplanned) says CLI clients cache managed plugin config without registering the marketplace or installing anything, while the 2.1.211 and 2.1.220 client bundles do contain managed "policy-required" plugin install/prune code. Treat the push as unproven until the pilot check passes: after first enabling, run `/plugin` on a pilot client and confirm the plugin appears with managed scope and its skills list. If auto-install does not happen on the deployed client version, the fallback is a one-time per-user `/plugin install <plugin>@<marketplace>` (announce it in the rollout note) — the managed `enabledPlugins` entry still prevents users from disabling it.

The Okta **groups claim is required** for per-group spend caps (`scope_type` `rbac_group`), which resolve against it — so the gateway requests the `groups` scope unconditionally. See `okta-request-email.md`.

Central push is a per-connected-client server-side control; it does **not** require or imply any admin rights on the laptop.

7. Which settings are user-scope, and which must be managed

An earlier rollout wrote a machine-wide `managed-settings.json`. Most of what it carried now lives in the user settings file or is compensated server-side — **but the two login keys genuinely cannot** and must come from a managed source. Claude Code offers the "Cloud gateway" login *only* when `forceLoginMethod: "gateway"` and `forceLoginGatewayUrl` are present in an **admin-controlled managed source** (§8). This is Anthropic's deliberate design — so a user can never be socially engineered into typing a hostile gateway URL that harvests their corporate SSO. Without the managed policy, `/login` shows the standard account picker with **no gateway option at all** — there is nothing for the developer to select, and no place to type a URL; a user-level `forceLoginMethod: "gateway"` in `~/.claude/settings.json` is ignored.

Old managed-settings key	What it did	Where it lives now
`forceLoginMethod: "gateway"`	Make the CLI offer/use gateway login	**Managed source only (§8) — no user-scope substitute exists.** Without it, `/login` has no gateway option at all. The network also blocks consumer `claude.ai`/Anthropic endpoints, but that does not create the login option; only the managed key does.
`forceLoginGatewayUrl`	Pre-fill the URL on the login screen (press Enter to connect)	**Managed source only (§8).** There is no user-facing way to type a gateway URL — by design.
`requiredMinimumVersion`	Refuse to start below a version floor	**Pushed centrally by the gateway** (§6e) — defaults to the gateway's own version, no GPO change needed. The key is managed-source-only; a GPO copy (§8) is possible but redundant, and pinning a *different* value there is undefined-precedence territory — leave it to the gateway push.
`env.DISABLE_UPDATES` / `env.DISABLE_AUTOUPDATER`	Lock auto-update	Written to the **user** settings `env` block by the installer; the gateway also pushes it centrally to every user via `/managed/settings` (§6); the mirror-only network path is the real control
`env.OTEL_RESOURCE_ATTRIBUTES` (`team` / `cost_center`)	Telemetry grouping	User settings `env` block (`-Team` / `-CostCenter`)
`env.NODE_EXTRA_CA_CERTS`	Enterprise CA trust	User settings `env` block (`-ExtraCaCertPath`)
(Okta auth, allowed email domains)	Who may use the gateway	**Gateway enforces Okta authentication + allowed email domains server-side** — never a client setting

So the model is **no-admin binary + user config, plus one required managed setting for login.** The network (only the gateway FQDN is reachable) and the gateway (rejects unauthenticated / wrong-domain / below-min-version clients) are compensations that *harden* the deployment, but they do **not** substitute for the login key — the "Cloud gateway" option simply does not exist on a client without it. The admin channel below is therefore required, not optional.

8. The managed-settings path for gateway login (required)

This is the required path for gateway login, not an optional "enforcement" add-on. Claude Code only exposes the "Cloud gateway" login when `forceLoginMethod: "gateway"` and `forceLoginGatewayUrl` are present in a **managed source**; delivering them also locks the method and pre-fills the URL so the developer just signs in (§3). `forceLoginMethod`, `forceLoginGatewayUrl`, and the optional `requiredMinimumVersion` are honored **only from a managed source**, and a managed source **overrides user and project settings** — so a developer cannot edit their way around them (nor edit their way *into* the gateway login without one).

Deliver them by **Group Policy** / **MDM** for a locked-down Windows fleet (the AD request is `ad-request-email.md`), by configuration management to the root-owned `/etc/claude-code/` file on **Linux** (§8.5), or self-serve them once on a machine where the developer has **local admin** (Windows) / **sudo** (Linux). The core value is the same small JSON on every OS (the two login keys; `parentSettingsBehavior` is an optional third). Two interchangeable Windows mechanisms follow; §8.5 is the Linux equivalent.

The managed-settings JSON to deliver (single object, one line for the registry value). `forceRemoteSettingsRefresh: true` makes the CLI block startup until it has freshly fetched the gateway's `/managed/settings` and **exit if that fetch fails** — which is what guarantees the model allowlist (§6) actually reaches the client instead of the client silently falling back to its built-in model menu. Trade-off: a gateway outage then stops Claude Code from starting at all.

```
{"forceLoginMethod":"gateway","forceLoginGatewayUrl":"https://<GATEWAY_FQDN>","forceRemoteSettingsRefresh":true}
```

Do **not** put `requiredMinimumVersion` in this JSON: the gateway pushes the version floor itself (§6e), defaulting to its own version — a GPO copy is redundant, and which managed source wins when both set a floor is undefined. (If your org deliberately manages the floor via GPO instead, set `MIN_CLIENT_VERSION=none` in `deploy.env` so only one source carries it.) There are two interchangeable delivery mechanisms; pick whichever fits the fleet's GPO conventions.

8.1 Mechanism A — GPP Registry item (recommended)

Deliver the settings as a single registry string value under the machine policy hive. Claude Code reads managed settings from `HKLM\SOFTWARE\Policies\ClaudeCode`, value name `Settings`, type `REG_SZ`, whose data is the one-line JSON above.

Steps an AD admin can follow:

1. In the Group Policy Management Console, edit a GPO linked to the OU containing the developer workstations (or a security group filtered to them).
2. Navigate to **Computer Configuration → Preferences → Windows Settings → Registry**. Machine-scope, so the policy applies regardless of which user logs on.
3. **New → Registry Item.** Set:
4. Action: **Update** (creates the value if missing, updates it if present — the safe default).
5. Hive: **HKEY_LOCAL_MACHINE**
6. Key Path: `SOFTWARE\Policies\ClaudeCode`
7. Value name: `Settings`
8. Value type: `REG_SZ`
9. Value data: the single-line JSON above, with `<GATEWAY_FQDN>` substituted.
10. Apply. Clients pick it up on the next Group Policy refresh (or `gpupdate /force`).

Use **Update** (not Replace) so the item is refreshed in place on each policy cycle without churn.

8.2 Mechanism B — GPP Files item (managed-settings.json)

Alternatively, deploy the same JSON as a file to the machine-wide managed path. Claude Code reads `%ProgramFiles%\ClaudeCode\managed-settings.json` — **not** `%ProgramData%` (the path moved at v2.1.75). `%ProgramFiles%` is admin-write-only, which is what makes the file tamper-resistant.

Steps:

1. Stage `managed-settings.json` (containing the JSON object above) on a share every client can read, e.g. `\\fileserver\software\claude\`.
2. In the GPO, navigate to **Computer Configuration → Preferences → Windows Settings → Files**. (Computer Configuration, so the destination resolves to the machine's `%ProgramFiles%` and the copy runs with machine rights — a standard user cannot write there.)
3. **New → File.** Set:
4. Action: **Update** (or Replace to overwrite on every refresh).
5. Source file(s): the UNC path, e.g. `\\fileserver\software\claude\managed-settings.json`.
6. Destination file: `%ProgramFiles%\ClaudeCode\managed-settings.json`.
7. Apply; clients copy the file on the next Group Policy refresh.

MDM equivalent: a device-scoped configuration profile (Intune Win32 app or a custom profile / script) that writes the same file to `%ProgramFiles%\ClaudeCode\` or the same value to `HKLM\SOFTWARE\Policies\ClaudeCode`. Deliver it in **device** context, not user context.

8.3 Why `HKCU\SOFTWARE\Policies\ClaudeCode` is not a no-admin backdoor

Anthropic's settings docs list `HKCU\SOFTWARE\Policies\ClaudeCode` as a real managed source (lowest policy priority), and a non-admin can write their own HKCU hive — so it's fair to ask whether a developer could self-serve the gateway login there without admin. **They cannot, for the login keys specifically:** the shipped build honors `forceLoginMethod` / `forceLoginGatewayUrl` only from source types `helper` / `plist` / `hklm` / `file` — `hklm` **but not** `hkcu`. So even on a machine where the `Policies` subtree is *not* ACL-locked, a user-written HKCU policy value is ignored for these keys. (On hardened fleets the `Policies` subtree is additionally GPO-locked / ACL-restricted under STIG/CIS baselines, so a non-admin can't write it at all.)

8.4 Upgrading from an earlier installer — clear stale managed settings

An earlier version of `Install-ClaudeCode.ps1` wrote forced-login keys (`forceLoginMethod` / `forceLoginGatewayUrl` / `requiredMinimumVersion`) to a **managed** source — `HKCU\SOFTWARE\Policies\ClaudeCode` on a non-admin run, or `%ProgramFiles%\ClaudeCode\managed-settings.json` (formerly `%ProgramData%`) when elevated. Because managed sources **override** the new user-scope settings, any of those left behind on a machine will keep taking effect after you switch to the current installer: a stale `forceLoginGatewayUrl` can lock the login screen to an old URL, and a stale `requiredMinimumVersion` can block an approved build. The current installer does **not** clean these up (it writes nothing to managed sources, so it has no basis to).

On a machine that was ever provisioned by the old installer, clear the stale managed settings **unless** you are deliberately taking them over via the GPO/MDM channel above. As the user (for the HKCU value) and as an admin (for the file):

```
Remove-Item -Path 'HKCU:\SOFTWARE\Policies\ClaudeCode' -Recurse -Force -ErrorAction SilentlyContinue
Remove-Item -Path (Join-Path $env:ProgramFiles 'ClaudeCode\managed-settings.json') -Force -ErrorAction SilentlyContinue
Remove-Item -Path (Join-Path $env:ProgramData 'ClaudeCode\managed-settings.json') -Force -ErrorAction SilentlyContinue  # pre-2.1.75 path
```

Then confirm with `/status` (below) that no unexpected managed source remains. Fresh fleets that never ran the old installer are unaffected.

8.5 Linux — the managed-settings file under `/etc/claude-code/`

On Linux (and WSL) Claude Code reads managed settings from the root-owned file `/etc/claude-code/managed-settings.json` — the same `file` source type as §8.2's `%ProgramFiles%` path, so the login keys are honored from it (§8.3's source-type list). `/etc` is root-write-only, which is what makes the file tamper-resistant: a developer without sudo cannot edit or remove it, and there is no user-scope location that can carry the login keys. The JSON is identical to the Windows delivery (pretty-printed is fine in a file):

```
{
  "forceLoginMethod": "gateway",
  "forceLoginGatewayUrl": "https://<GATEWAY_FQDN>",
  "forceRemoteSettingsRefresh": true
}
```

Deliver it with whatever manages your Linux fleet — Ansible/Salt/Puppet copying the file (owner `root:root`, mode `0644`: world-readable so the CLI can read it from any account, root-write-only so no one else can change it) — or set it once by hand on a machine where the developer has sudo:

```
sudo mkdir -p /etc/claude-code
sudo tee /etc/claude-code/managed-settings.json >/dev/null <<'EOF'
{
  "forceLoginMethod": "gateway",
  "forceLoginGatewayUrl": "https://<GATEWAY_FQDN>",
  "forceRemoteSettingsRefresh": true
}
EOF
sudo chmod 644 /etc/claude-code/managed-settings.json
```

(The portal's Linux installer prints a self-serve snippet with the two login keys and the refresh flag — the deployment's real gateway URL substituted — at the end of every install. As on Windows, do **not** add `requiredMinimumVersion` here: the gateway pushes the version floor itself — see the §8 intro.)

Anthropic's settings docs also support a drop-in directory (`/etc/claude-code/managed-settings.d/*.json`) for layering additional managed fragments; this deployment's runbooks use only the single file. Verification is the same as Windows: `/status` inside `claude` must show the login keys sourced from the managed layer (§8.6).

8.6 Verifying the active configuration

Inside `claude`, run `/status` — it shows the **active setting sources**, so an admin can confirm the managed source is present and winning over user settings. Precedence, highest first: managed source (GPO/MDM) → project settings → user settings (`%USERPROFILE%\.claude\settings.json`). A `forceLoginMethod` shown as sourced from the managed layer confirms the lock is in force.

---

9. Summary — three channels

- **Installer + user settings (no admin):** the binary, PATH, telemetry tags, update lockdown, and enterprise CA trust — everything except login, entirely in user scope for the whole fleet with zero elevation.
- **Managed settings for login (admin — REQUIRED):** `forceLoginMethod:"gateway"`
- `forceLoginGatewayUrl` (leave `requiredMinimumVersion` to the gateway push, §6e), delivered by GPO/MDM (`ad-request-email.md`) or self-served with local admin. Without this the gateway login does not exist on the client; with it, login is method-locked and URL-prefilled (§3). This is the one part that is not admin-free.
- **Gateway** `/managed/settings` **(server-side):** the **client model allowlist** (`availableModels` / `enforceAvailableModels`), the web/MCP tool denies, the small/fast-model override, the **minimum client version floor** (`requiredMinimumVersion`, defaulting to the gateway's own version), optional **organization-wide Claude rules** (`claudeMd` from `CLAUDE_MD_FILE`, §6g), the optional **session-recap disable** (`DISABLE_SESSION_RECAPS`, §6h), optional **enterprise skills** as force-installed plugins (`PLUGIN_MARKETPLACE_*` / `MANAGED_PLUGINS`, §6i), and central telemetry config + update lockdown for every connected client.

The channels compose cleanly and target different keys: the installer writes no policy source, the managed-settings channel owns forced login, and the gateway push owns the model allowlist plus central telemetry/update lockdown.